HACKSIMUL8 2026 — 6-DOF UAV Quadcopter Control

PID altitude control with ML-based self-tuning of controller gains

Department of Mechanical Engineering, PES University · in partnership with MathWorks 2 September 2026

The problem

Design a control system for a 6-DOF UAV quadcopter, in four tasks:

1. **Model** — dynamic mathematical model in state-space form, MATLAB and Simulink
2. **Control** — altitude control using PID, with a justified tuning methodology
3. **Learn** — generate simulation data and train an ML model for self-tuning of PID gains
4. **Test** — implement the model and present test cases with comparisons

Reference: Mien, T. & Tu, T. (2024), "*Design and Quality Evaluation of the Position and Attitude Control System for 6-DOF UAV Quadcopter Using Heuristic PID Tuning Methods*", IJRCS 4(4), 1712–1730. doi:10.31763/ijrcs.v4i4.1594

Status

Task	Status
1 — Dynamic model, state space	✅ **Complete, verified** (25/25 checks)
2 — PID altitude control	✅ **Complete, benchmarked against the cited paper**
3 — Data generation + ML model	✅ **Complete**
4 — Test cases and comparison	✅ **Complete** — mean +61.2% ITAE improvement

Headline results

Metric	Value
Model verification `max|A - A_numerical|`	**1.636e-12**
Controllability	**rank 12 of 12**
Final PID gains	**Kp = 30.1847, Ki = 0.0000, Kd = 10.3994**
Settling time	**0.98 s**
Overshoot	**0.00 %**
Altitude sag at 17.2° pitch	**0.000134 m**
ML self-tuner, mean ITAE improvement	**+61.2%** (four of five cases +70–82%)

Benchmarked against the reference paper

All four designs simulated on the identical plant, stepping to the paper's own setpoint of z = 2.0 m. The three from Mien & Tu run as plain PID (their Eq. 27); ours runs feedback-linearised.

Design	Kp	Ki	Kd	Ts [s]	OS [%]	ITAE	Tilt sag [m]
Ziegler–Nichols (Mien & Tu)	74.994	42.610	32.997	4.610	12.13	1.7259	0.004522
Tyreus–Luyben (Mien & Tu)	56.814	7.337	31.744	10.300	4.96	6.6841	0.034002
MATLAB PID Tuner (Mien & Tu)	57.005	0.001	23.102	1.490	0.00	0.3319	0.008033
Ours (ITAE + feedback lin.)	**30.185**	**0.000**	**10.399**	**0.980**	**0.00**	**0.1874**	**0.000134**

Lowest ITAE of all four — 1.8× better than the best of their three, 36× better than Tyreus–Luyben, which their paper concludes is best. And **253× less altitude sag** under a 17.2° pitch step.

Independent check on our reproduction: their paper reports Tyreus–Luyben achieving "steady-state error of less than 1%". Our simulation of their published gains gives **0.592%** — inside their stated bound.

What makes this submission different

1. Feedback linearisation on the altitude channel. Commanding `U1 = m(g + u_z)/(cos ϕ cos θ)` makes `z̈ = u_z` *exactly*, at any tilt angle — not a small-angle approximation. The reference paper's Eq. (27) is a plain PID with no gravity feed-forward and no tilt compensation.

2. The controller is never told the payload mass. In steady hover thrust must equal weight, so mean thrust ÷ nominal weight **is** the mass ratio (r = +0.840 against optimal Ki). Most "self-tuning" implementations feed the mass in as a model input — that is gain scheduling, not self-tuning.

3. We report what the data can and cannot identify. Ki predicts at $R^2 \approx 0.70$–0.87; Kp and Kd do not, because the cost surface has a flat valley in those directions and their labels are intrinsically noisy. Adapting only the identifiable parameter is the correct engineering response, and we say so rather than hiding it.

Quick start

```
cd solution
verify_all                      % 25 independent checks, ~2 min
open_system('quad_altitude')    % press Run, then open the Scope
task2b_zn_tyreus_luyben         % the reference-paper benchmark
```

`verify_all` **re-derives** every result rather than loading saved ones, so it is genuine evidence rather than a replay.

Documentation

File	Contents
RUNBOOK.md	**How to run everything; every figure explained — which script made it and how**
README_HANDOFF.md	Index, file guide, headline results
TASK1_final.md	Model derivation, state space, verification, presentation script, judge Q&A
TASK2_final.md	Feedback linearisation, four tuning methods, paper benchmark, Q&A
TASK3_final.md	Dataset design, model selection, defects found and fixed, Q&A
TASK4_final.md	Self-tuner implementation, five test cases, why only Ki is adapted, Q&A

Every document also exists as a PDF.

Repository layout

```
├── RUNBOOK.md/.pdf          how to run everything, every figure explained
├── README_HANDOFF.md/.pdf   index and file guide
├── TASK1_final.md/.pdf      model + verification
├── TASK2_final.md/.pdf      PID + tuning methodology
├── TASK3_final.md/.pdf      ML dataset + model
├── TASK4_final.md/.pdf      self-tuner test cases and comparison
├── solution/                18 MATLAB files, Simulink model, results, figures
│   ├── quad_params.m        all Table 1 parameters (single source of truth)
│   ├── quad_dynamics.m      nonlinear 6-DOF equations
│   ├── task1_statespace.m   builds and verifies A, B, C, D
│   ├── task2_altitude_pid.m four tuning methods
│   ├── task3_*.m            ML data generation and training
│   ├── task4_*.m            self-tuner test cases
│   ├── verify_all.m         25-check verification suite
│   ├── quad_altitude.slx    the Simulink model
│   └── figures/             7 presentation figures, 150 dpi
├── reference/               the Mien & Tu paper + problem statement
├── frontend_assets/         structured data.json + figures for a web write-up
└── archive/                 superseded drafts and course notes
```

Requirements

MATLAB **R2026a** with Simulink, Control System Toolbox, Deep Learning Toolbox, Simulink Control Design, Stateflow. Parallel Computing Toolbox is optional — Tasks 3 and 4 fall back to serial execution without it, roughly 4× slower.

The MATLAB installer is **not** included in this repository — it is proprietary MathWorks software. Download it from mathworks.com/downloads with your own licence.

Parameters (Table 1, given)

Parameter	Symbol	Value
Quadcopter mass	m	0.516 kg
Arm length	l	0.225 m
Gravity	g	9.81 m/s^2
Rotor inertia moment	I_M	3.368×10^{-5} $kg\cdot m^2$
Thrust factor	k	2.996×10^{-6} $N\cdot s^2$
Drag coefficient	b	1.260×10^{-7} $N\cdot m\cdot s^2$
Inertia	Ixx, Iyy	4.984×10^{-3} $kg\cdot m^2$
Inertia	Izz	8.958×10^{-3} $kg\cdot m^2$

Derived: hover weight **W = 5.0620 N**, hover rotor speed **649.9 rad/s** each, max thrust **20.248 N**, max upward acceleration **29.43 m/s^2** (≈ 3 g).